Final Year Project Report
Smart Scheduler

Project Advisor: Amjad Ali

Submitted By:

Sibgha Zeeshan (F2021266550)
Areeba Noor (F2021266503)
Rabia Khalid (F2021266534)
Mahak Khalil (F2021266627)

(Session 2021-2025)

University of Management and Technology
C-II Johar Town Lahore Pakistan

Dedication

The project is dedicated to academic researchers and professionals in the field of artificial intelligence and automated scheduling. Their groundbreaking work and dedication to innovation have been a source of inspiration for us in developing this project. We also dedicate this work to our institution and mentors, whose guidance and support have enabled us to transform ideas into practical outcomes.

Final Approval

- **Head of Department** _____
  Department of Computer Science.
  School of Systems & Technology
  UMT Lahore

- **Director (Final Year Projects-CS)** _____
  Department of Computer Science.
  School of Systems & Technology
  UMT Lahore

- **Supervisor** _____
  Department of Computer Science.
  School of Systems & Technology
  UMT Lahore

- **Co-Supervisor** _____

Acknowledgment

We express our sincere appreciation to our project advisor, Amjad Ali, for his invaluable guidance, constructive feedback, and continuous support throughout. His insights and expertise have been crucial in shaping the Smart Scheduler. We also acknowledge the efforts of the faculty members of the Department of Computer Science, UMT Lahore, for providing us with the necessary resources.

Project Title:

Smart Scheduler

Objective:

The project establishes an automated university timetable generator which applies Constraint Satisfaction Problems (CSP) for intelligent scheduling functions. The system provides efficient scheduling functionality alongside optimal resource management, and it handles conflicts among faculty scheduling and room availability and course schedules.

Undertaken by:

Sibgha Zeeshan (F2021266550)

Areeba Noor (F2021266503)

Rabia Khalid (F2021266534)

Mahak Khalil (F2021266627)

Supervised by:

Amjad Ali

Starting Date:

November 2024

Completion Date:

July 2025

Tools Used:

- **Programming Languages:** Python for backend development and TypeScript for frontend.
- **FrameWorks:** Next.js for frontend development. FastAPI for backend.
- **Libraries:** Pandas, PyMongo and Numpy for implementing AI algorithms.
- **Database:** MongoDB for managing scheduling data and user information.
- **Development Environment:** Integrated Development Environment (IDE) such as VS Code.
- **Deployment:** Docker, Railway and Vercel.
- **Version Control:** Git for source code management and version control.

Abstract

The Smart Scheduler uses automated algorithms to generate academic schedules which resolve complex scheduling issues in educational institutions. Academic schedules still use manual methods that take a considerable amount of time and cause conflicts which result in poor resource use. The goal of this project is to solve current scheduling problems using Constraint Satisfaction Problems (CSP) and AI-based optimization algorithms.

The system uses automatic algorithms to schedule all resources effectively and eliminate conflicts that arise between them. The system delivers scalability together with adaptability and optimal resource management capabilities that match universities along with colleges and other educational institutions. Through the implementation of the Smart Scheduler organizations cut both scheduling costs and increase alignment with institutional requirements and dynamic events.

AI-based scheduling methods enable the computer program to create conflict-free schedules that optimize faculty workload allocations and assign classrooms effectively. The system development follows modern programming languages as well as AI frameworks and database management systems to create resilient operations.

Educational institutions achieve both efficient error-free automated timetable management and improved academic operations through the Smart Scheduler which eases administrative workload.

Table of Contents

Contents **7**
Definitions and Acronyms 10
List of Figures 11
List of Tables 12
1. Introduction **13**
1.1 Motivation 13
1.2 Project Overview 13
1.2.1 Problems or Overview Statement 13
1.2.2 Customer 13
1.2.3 Goals 14
1.2.4 System Functions 14
1.2.5 System Attributes 14
1.3 Problem Statement 14
1.4 Objectives 15
2. Domain Analysis **17**
2.1 Customer 17
2.2 Stakeholder 17
2.3 Affected Groups with social or economic impact 18
2.4 Dependencies/ External Systems 18
2.5 Reference Documents 18
2.5.1 Related Projects 18
2.5.2 Literature Review 18
3. Requirement Analysis **21**
3.1 Requirements 21
3.2 List of Actors 22
3.3 List of Use Cases 22
4. System Design **23**
4.1 System Use Case Diagram 23
4.2 Class Diagram 24
4.3 Sequence Diagram 25
4.4 Flow Diagram 26
4.5 ERD 27

5. Methodology 29
5.1 Suggested Approach 29
5.2 Design and Procedures 29
5.3 Constraints Being Resolved 29
5.3.1 Hard Constraints 29
5.3.2 Soft Constraints 30
5.4 Sources of Data and Sampling Procedures 30
5.4.1 Input Data Structure 30
5.5 Workflow of the System 32
5.6 Output Generated 33
5.6.1 Main Timetable Sheet 33
5.6.2 Unassigned Courses Sheet(if any conflicts occur) 33
5.7 Key Features 33
5.8 Limitations and Assumptions 34
5.8.1 Assumptions 34
5.8.2 Limitations 34
6. Implementation Details 35
6.1 Development Setup 35
6.2 Deployement Setup 36
6.3 Algorithms 36
6.3.1 Faculty Course Assignment Algorithm (Heuristic Rating-Based Selection) 36
6.3.2 Time Slot and Room Precomputation Algorithm 37
6.3.3 Constraint Bases Timetable Scheduling Algorithm(Bactracking + Heuristic) 37
6.3.4 Conflict Detection Algorithm 38
6.3.5 Scheduling Timeout and Relaxed Retry Mechanism 38
6.4 Genetic Algorithm 39
6.4.1 Code Structure and Flow 39
6.5 System Constraints 44
6.6 Assumptions 46
6.7 Restrictions 46
6.8 Limitations 47
7. Testing 48
7.1 Extended Test Cases 48
7.2 Decision Table 54
7.2.1 Code Snippet 54
7.2.2 Decision Coverage Table 55

Table 7-1: Decision Coverage Table 55
7.2.3 RID Definitions 56
7.2.4 UCID Definitions 56
7.2.5 TID Definitions 57
7.2.6 RID vs UCID (Requirements vs Use Cases) 58
7.2.7 RID vs TID (Requirements vs Test Case ID) 59
7.2.8 UCID vs TID (Use Cases vs Test Case ID) 59
8. Results 60
8.1 Completion 60
8.2 Accuracy 60
8.3 Correctness 60
9. Conclusion 61
10. Future Work 62
11. Bibliography 63
11.1 Books 63
11.2 Journals 63
11.3 Articles 63
11.4 Research papers 63
12. Appendix 64
12.1 Glossary of Terms 64

Definitions and Acronyms

Acronym	Definition
UMT	University of Management and Technology
CSP	Constraint Satisfaction Problem
AI	Artificial Intelligence
IDE	Integrated Development Environment
AWS	Amazon Web Services
SQL	Structured Query Language
VS Code	Visual Studio Code
CSV	Comma-Separated Values
ERD	Entity Relationship Diagram
UML	Unified Modeling Language

Table 00-1: Table of acronyms and definitions

List of Figures

Figure 4-1: System Use Case Diagram 23
Figure 4-2: Class Diagram 24
Figure 4-3: Sequence Diagram 25
Figure 4-4: Flow Diagram 26
Figure 4-5: ERD 27
Figure 5-1: Data 32

List of Tables

Table 00-1: Table of acronyms and definitions 10
Table 2-1: List of Stakeholders 17
Table 2-2: Literature Review 20
Table 3-1:Requirements 21
Table 3-2: Requirement Analysis 21
Table 7-1: Decision Coverage Table 55
Table 7-2: RID Definitions 56
Table 7-3: UCID Definitions 56
Table 7-4: TID Definitions 57
Table 7-5: RID vs UCID matrix 58
Table 7-6: RID vs TID 59
Table 7-7: UCID vs TID 59
Table 12-1: Glossary 64

1. INTRODUCTION

1.1 Motivation

The Smart Scheduler project effectively tackles the scheduling problems occurring during manual timetable development at educational institutions. The process of scheduling lectures, exams, labs and evaluating faculty availability, room capacity and student schedules proves to be complicated and results in scheduling conflicts and room inefficiencies. The smart scheduler operates to automate tasks and cut down human mistakes as well as administrative tasks. The system implements algorithms to achieve balanced class distribution and avoid scheduling conflicts and maximize resource efficiency. The system needs to accommodate rising student numbers and increasing courses because scalability and adaptability will determine its effectiveness. Through its implementation the Smart Scheduler offers efficiency together with reliability and adaptability to create an automated academic scheduling process that simplifies user interactions.

1.2 Project Overview

The timetable scheduling problem requires organizing academic activities together with the allocation of limited resources which include classroom instructors and time slots. The main challenge involves building a schedule which prevents resource double-booking and class time collisions without neglecting students' and faculty's specific requirements. As institutions expand their operations become more difficult because they need to schedule courses and instructors alongside more students. The combination of different scheduling constraints which include room scheduling and instructor selection and student timetable requirements creates an NP complete status for the timetable scheduling problem with its optimization task classified as NP Hard.

1.2.1 Problems or Overview Statement

The manual generation of academic timetables proves to be a difficult process which frequently results in scheduling problems and suboptimal classroom assignments while causing resources to be underutilized. The Smart Scheduler system has automated scheduling to achieve optimal results and decrease manual handling.

1.2.2 Customer

The primary users of this system are educational institutions, including universities, colleges, and schools, where academic scheduling plays a crucial role in daily operations. Administrators, faculty members, and students will benefit from a more streamlined and effective timetable management system.

1.2.3 Goals

- Develop a system that generates optimized timetables with minimal conflicts.
- To reduce administrative workload by automating the scheduling.
- Efficiently manages the distribution of classrooms together and faculty availability.
- Provide a scalable solution adaptable to different institutions and scheduling needs.
- The system should enable users to conduct changes and updates through an easy-to-use interface that delivers an optimized experience.

1.2.4 System Functions

- The system generates schedules by applying predefined limitations to its operations.
- Conflict detection and resolution.
- Resource allocation optimization (rooms, faculty, time slots).
- User-friendly interface for schedule adjustments and overrides.
- The system implements reporting tools for scheduling efficiency analysis as well as analytical tools for tracking scheduling performance.

1.2.5 System Attributes

- **Scalability** – Capable of handling increasing numbers of students and courses.
- **Reliability** – Ensures accurate and conflict-free scheduling.
- **Flexibility** – Allows manual adjustments when necessary.
- **User-Friendliness** – Intuitive interface for administrators and faculty.
- **Efficiency** – Reduces time and effort required for scheduling processes.

The Smart Scheduler project is designed to revolutionize academic scheduling by eliminating inefficiencies and providing a reliable, automated solution for educational institutions.

1.3 Problem Statement

Manual scheduling systems consume a lot of time while producing scheduling issues that result in overbooked classrooms and empty slots in the schedule. The project targets complete scheduling automation through the resolution of hard restrictions such as schedule conflicts and soft limits regarding schedule gaps to create optimized timetables that adhere to institution requirements. The solution becomes complicated because it requires handling several scheduling constraints which include classroom conflicts and restricted spaces and varied class session periods as well as professor scheduling priorities. The current static scheduling systems fail to adapt to real-time modifications resulting in resource inefficiencies which demonstrates the worth of developing this solution.

1.4 Objectives

AI algorithms based on Constraint Satisfaction Problems (CSP) together with combinatorial search techniques hold great potential to solve the scheduling problem which falls under the NP-complete and NP-hard classification. These methods enable institutions to automate and optimize scheduling, accounting for both essential constraints (e.g., avoiding double-booking) and preferential constraints (e.g., minimizing gaps in the timetable). By applying these techniques, the project aims to generate efficient timetables that improve resource management, enhance academic experience, and reduce the administrative burden. Study of multiple Artificial Intelligence approaches will create a smarter operational system which is of significant value in today's fast-evolving educational landscape.

The goal of this project is to develop an AI-based timetable scheduler that automates academic schedule organization for lectures, examinations and laboratory sessions. The system effectively handles essential resources which include classrooms, instructors, students and time slots by resolving operational blocking constraints and soft constraints while minimizing time gaps. The system will implement sophisticated AI approaches consisting of Constraint Satisfaction Problems (CSP) and combinatorial search methods to generate optimal timetables quickly. Educational organizations that implement this system can look forward to obtaining better resource distribution alongside decreased scheduling problems and increased scheduling adaptability. Automation in scheduling helps lower manual labor which enables administrators to handle strategic planning instead of routine work. The product delivers an intelligent scalable solution which provides users with a user-friendly interface.

Upon completion of this project, the Smart Scheduler is expected to achieve the following objectives:

- **Precise allocation of classrooms** to prevent overcrowding and ensure optimal usage of available spaces.
- **Automated conflict resolution** to eliminate clashes in scheduling for faculty, students, and rooms.
- **Dynamic timetable adjustments** to accommodate unforeseen changes, such as faculty unavailability or room constraints.
- **Optimization of faculty workload distribution** to prevent overburdening and ensure fair allocation of teaching hours.
- **Scalable scheduling framework** that can adapt to increasing student enrollment and course expansions.
- **Enhanced user interface for easy schedule modifications** by administrators and faculty members.
- **Data analytics and reporting** provide insights into scheduling efficiency, resource utilization, and potential improvements.

- **Seamless integration with existing institutional systems** to streamline data exchange and minimize redundant data entry.
- **Implementation of AI-driven scheduling algorithms** to maximize efficiency and minimize human intervention.

By achieving these objectives, the Smart Scheduler will significantly improve the efficiency and effectiveness of academic scheduling, benefiting administrators, faculty, and students alike.

2. DOMAIN ANALYSIS

2.1 Customer

Educational institutions including universities, colleges and schools comprise the main user base of the Smart Scheduler system. Academic institutions need a performance-oriented automated scheduling platform to operate their academic timetables. The system exists to deliver a dependable conflict-free scheduling tool which serves administration staff and faculty members and student bodies.

2.2 Stakeholder

This table identifies the primary stakeholders of the system and briefly describes their respective roles and responsibilities. It highlights how each stakeholder contributes to the system's functionality and efficiency.

Stakeholder	**Role in System**
University Administrators	Oversee timetable creation, manage faculty assignments, and ensure efficient resource utilization.
Class Coordinators	Allocate classrooms, monitor schedule conflicts, and request reports on missed classes or clashes.
Faculty Members	View and manage their assigned schedules, request changes, and report conflicts.
Students	Access their course schedules and receive real-time updates on any changes.
IT Support Team	Maintain and support the system, ensuring its functionality and security.

Table 2-1: List of Stakeholders

2.3 Affected Groups with social or economic impact

- **Faculty Members**: Fair workload distribution and minimized scheduling conflicts improve job satisfaction.
- **Students**: Better timetable management reduces clashes and ensures a smooth learning experience.
- **Administrative Staff**: Reduced workload and enhanced efficiency in timetable management.
- **Institutional Management**: Optimized resource utilization leads to cost savings and improved institutional performance.

2.4 Dependencies/ External Systems

The Smart Scheduler system may rely on various external systems for its operation, including:

- **University Management Systems (UMS)** – For integrating student and faculty data.
- **Room Booking Systems** – To verify and allocate available classrooms efficiently.
- **Email/Notification Services** – For real-time updates and alerts to users.
- **Cloud Storage & Databases** – For securely storing schedules and related data.

By incorporating these dependencies, the Smart Scheduler ensures a seamless and efficient scheduling process while integrating with existing institutional infrastructure.

2.5 Reference Documents

2.5.1 Related Projects

- Celcat Timetabler
- UniTime
- ASC Timetables
- Optime Schedule

2.5.2 Literature Review

This section presents a review of existing research and systems related to timetable scheduling. It provides insights into previous approaches, methodologies, and technologies used to address scheduling challenges.

Sr.no	Title & Author of Research Paper	Publication Details	Methodology & Solution	Limitations
1.	**Genetic Algorithm to Generate the Automatic Time-Table – An Overview** Authors: Asif Ansari, Prof. Sachin Bojewar	*International Journal on Recent and Innovation Trends in Computing and Communication*, Volume 2, Issue 11, November 2014	Utilizes a genetic algorithm (GA) framework where solutions (chromosomes) are optimized using crossover, mutation, and fitness evaluation. Hard constraints (e.g., avoiding overlapping classes) and soft constraints (e.g., scheduling preferences) are considered to generate optimal timetables, reducing manual scheduling efforts.	The system may get stuck in local optima and struggles with extreme constraint scenarios.
2.	**University Course Timetable Using Constraint Satisfaction and Optimization** Author: Tarek El-Sakka	*International Journal of Computing Academic Research*, Volume 4, Number 3, June 2015	Uses Constraint Logic Programming (CLP) and Optimization Programming Language (OPL) to solve the timetabling problem. Models the issue as a Constraint Satisfaction Problem (CSP) and employs IBM ILOG CPLEX Studio to optimize resource allocation while addressing hard and soft constraints. Tested on real-world university data.	Challenges in modeling complex constraints and dependence on high computational resources.
3.	**Automated Timetable Generator** Authors: Prof. Er. Shabina Sayed et al.	*International Journal for Innovative Research in Science & Technology*, Volume 1, Issue 11, April 2015	Uses evolutionary algorithms (EAs) for timetable generation. Data on subjects, teachers, classrooms, and preferences are processed through a genetic algorithm to optimize schedules while maintaining constraints. A verification phase ensures compliance with scheduling rules.	Struggles with highly complex constraints and requires manual intervention for final verification.
4.	**Automatic Timetable Generation Using Genetic Algorithm** Authors: Dipesh Mittal, Hiral	*International Journal of Advanced Research in Computer and Communication Engineering*,	Implements a Genetic Algorithm (GA)-based approach to solve the NP-hard scheduling problem. Generates an initial population of schedules, evaluates fitness, and refines	Computational overhead, parameter tuning challenges, and risk of premature convergence.

	Doshi, Mohammed Sunasra, Renuka Nagpure	Volume 4, Issue 2, February 2015	solutions through crossover and mutation. Effectively handles hard constraints (avoiding teacher/class/room overlaps) and soft constraints (minimizing free periods for students).	
5.	**Automated Timetable Generation Using Machine Learning Algorithms** Authors: Shraddha Ambhore et al.	*International Journal of Research in Engineering, Science and Management*, Volume-3, Issue-3, March 2020	Uses evolutionary and heuristic algorithms (Genetic Algorithm, Tabu Search, Simulated Annealing, and Scatter Search) to optimize scheduling. Factors such as faculty workload and university constraints are considered, and fitness functions are defined for optimization.	Time-intensive parameter tuning and difficulty in ensuring high-quality solutions for complex constraints.
6.	**Timetable Generator** Authors: Swapnil Markal, Surekha Ghorpade, Diksha Chalke	*IOSR Journal of Computer Engineering*, Volume 22, Issue 2, March-April 2020	Employs Genetic Algorithm (GA) for timetable generation, encoding parameters such as teacher names, subjects, and room availability. Optimized using selection, crossover, and mutation, ensuring adherence to hard constraints (avoiding scheduling conflicts) and optimizing soft constraints (reducing consecutive lectures).	Requires accurate input data and struggles with dynamic schedule changes, requiring recomputation.
7.	**Student Scheduling at Purdue University** Author: Tomáš Müller	PATAT 2024 Conference	Describes the UniTime system for student scheduling at Purdue University. Uses a hybrid approach combining constraint programming, local search, and optimization heuristics (branch-and-bound, stochastic hill climbing).	Requires significant computational power and ongoing refinement to manage complex dynamic interactions.

Table 2-2: Literature Review

3. REQUIREMENT ANALYSIS

3.1 Requirements

This section outlines the system requirements, including functional, non-functional, data-related, and constraint-based needs. It defines what the system must do, how it should perform, and the limitations it must operate within.

Functional Requirements	Core system functionalities like schedule generation, conflict resolution, and faculty-student schedule management.
Non-Functional Requirements	Performance, security, and scalability aspects of the system.
Data Requirements	How scheduling data, faculty, and room assignments will be stored and accessed.
Constraints	Limitations such as security policies, role-based access, and compliance requirements.

Table 3-1:Requirements

Below is the table outlining the functional and non-functional requirements of the Smart Scheduler system.

RID	Description	Category	Attribute	Details and Boundary Constraints
FR_1	Generate optimized class timetable	Functional Requirement	Response Time	Schedule should be generated in less time.
FR_2	Allocate classrooms based on availability	Functional Requirement	Resource Allocation	Rooms must be assigned efficiently without conflicts.
NFR_1	Provide real time schedule update	NonFunctional Requiremnet	System Performance	Notifications should be sent instantly upon changes.
DR_1	Securely store and retrieve scheduling data	Data Requirement	Data Security	Must comply with institutional data security policies.
C_1	Ensure compliance with academic regulations	Constraint	Policy Compliance	Scheduling rules must align with university policies.

Table 3-2: Requirement Analysis

3.2 List of Actors

- **Admin/Registrar**: Manages timetable generation, resolves conflicts, allocates resources, and oversees system operations.

- **Faculty**: Registers their department, shows preferences for scheduling, and provides feedback on the generated timetable.

- **Student**: Views and downloads the generated timetable.

3.3 List of Use Cases

- **Login** – Allows authorized users (Admin/Registrar, Faculty) to access the system.
- **Validate Timetable** – Ensures the generated timetable is correct and meets all constraints.
- **Register Department** – Faculty registers their department in the system.
- **Show Preference** – Faculty provides preferences for scheduling (e.g., time slots, subjects).
- **Give Feedback** – Faculty provides feedback on the generated timetable.
- **Allocate Resources** – Admin assigns rooms and resources to classes.
- **Convert File** – Converts the generated timetable into a downloadable format.
- **Generate Timetable** – The system automatically creates a timetable based on constraints.
- **Resolve Conflicts** – Admin resolves scheduling conflicts.
- **Download Timetable** – Students can download the finalized timetable.
- **View Timetable** – Students can view the timetable online.

4. SYSTEM DESIGN

4.1 System Use Case Diagram

The following use case diagram illustrates the interactions between users (Admin, Registrar, Faculty, and Student) and the Smart Scheduler system. It highlights key system functionalities such as timetable generation, resource allocation, and user preferences.

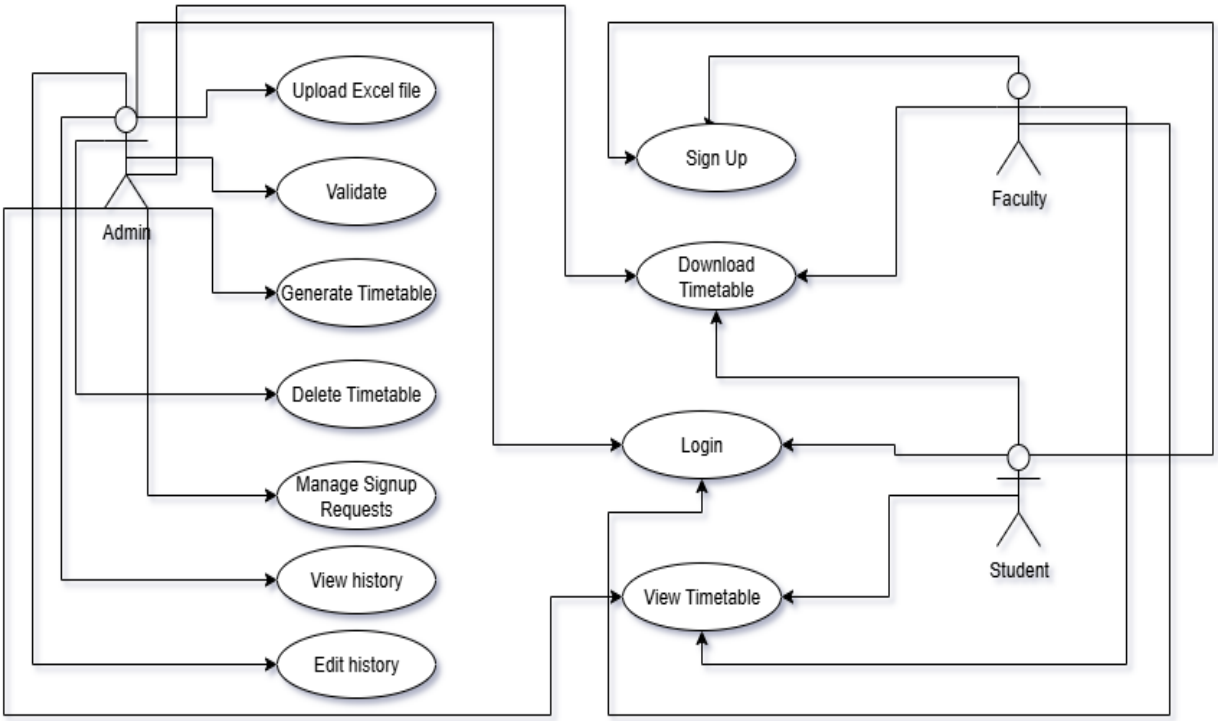

Figure 4-1: System Use Case Diagram

4.2 Class Diagram

The following class diagram represents the static structure of the Smart Scheduler system. It shows the main system classes, their attributes, methods, and the relationships between them such as associations and inheritance.

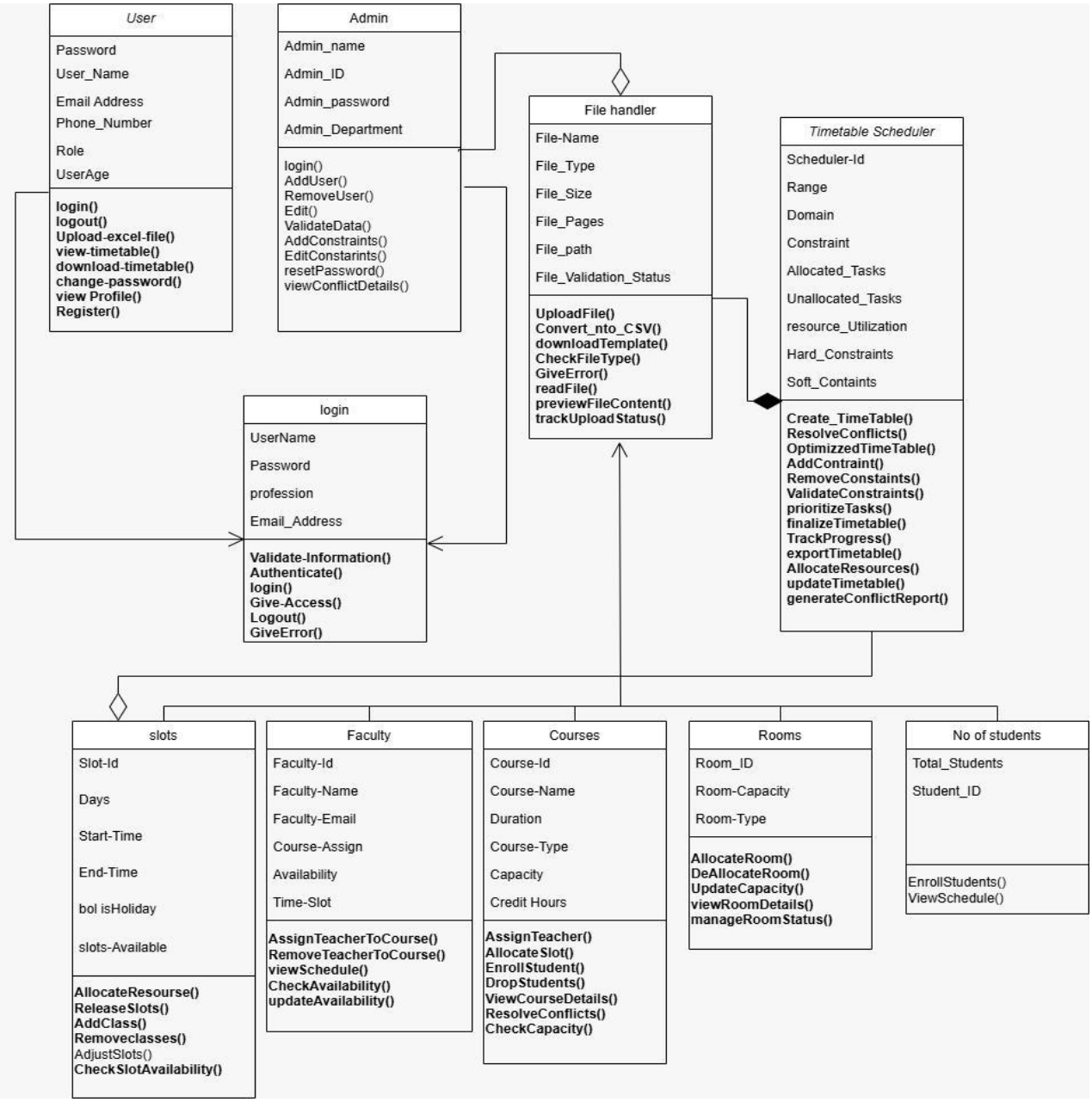

Figure 4-2: Class Diagram

4.3 Sequence Diagram

The sequence diagram depicts the interaction flow between system components and users in the Smart Scheduler. It illustrates the order of operations for generating and accessing the timetable, including login, preference submission, and timetable generation.

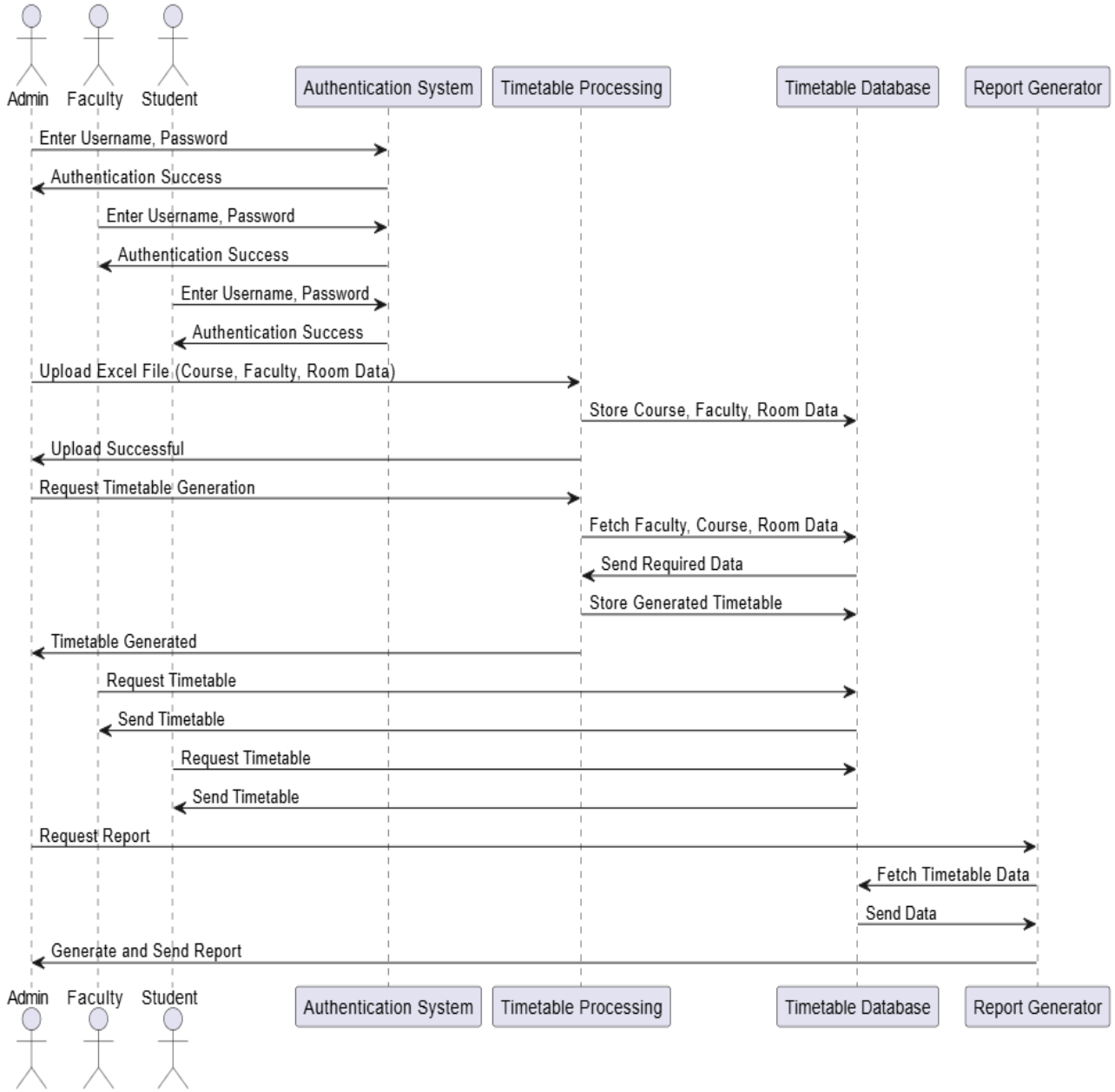

Figure 4-3: Sequence Diagram

4.4 Flow Diagram

The flow diagram outlines the step-by-step process of the Smart Scheduler system. It visually represents the logical flow of operations from user login and data input to timetable generation, conflict resolution, and final timetable distribution.

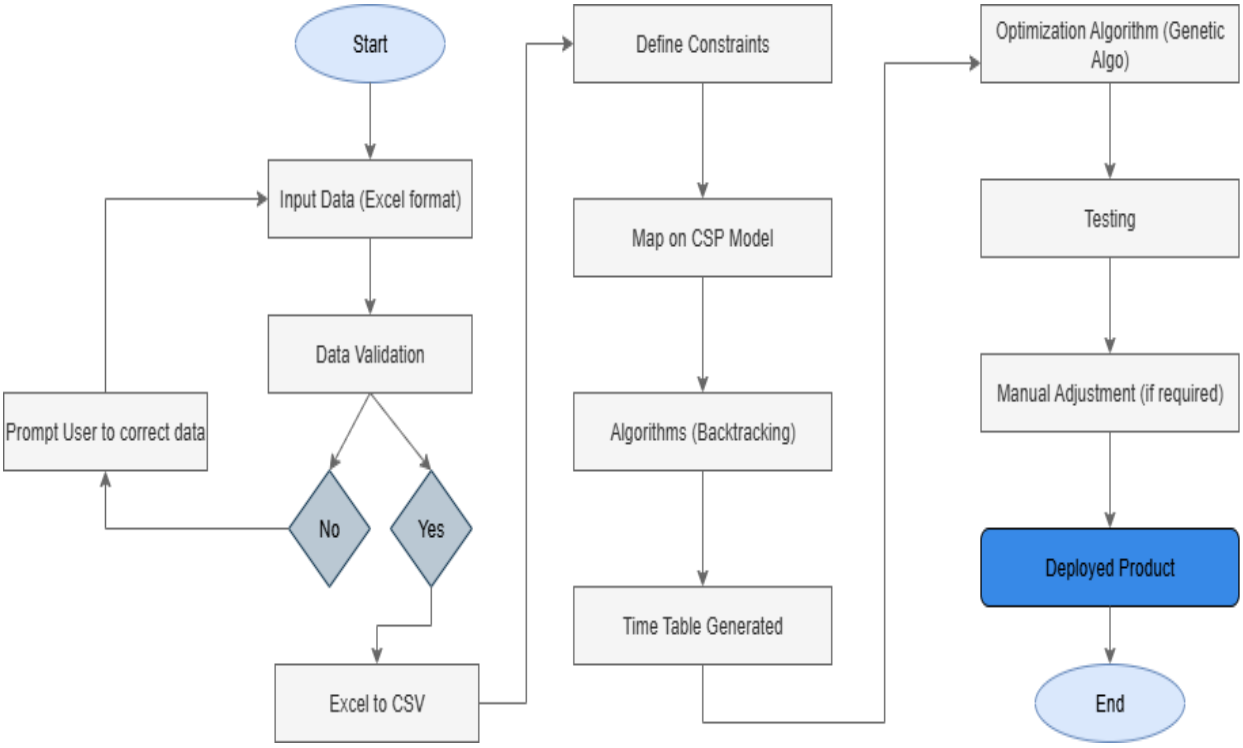

Figure 4-4: Flow Diagram

4.5 ERD

The Entity-Relationship Diagram (ERD) represents the database structure of the Smart Scheduler system. It defines the key entities, their attributes, and the relationships between them, such as students, faculty, courses, rooms, and timetable allocations.

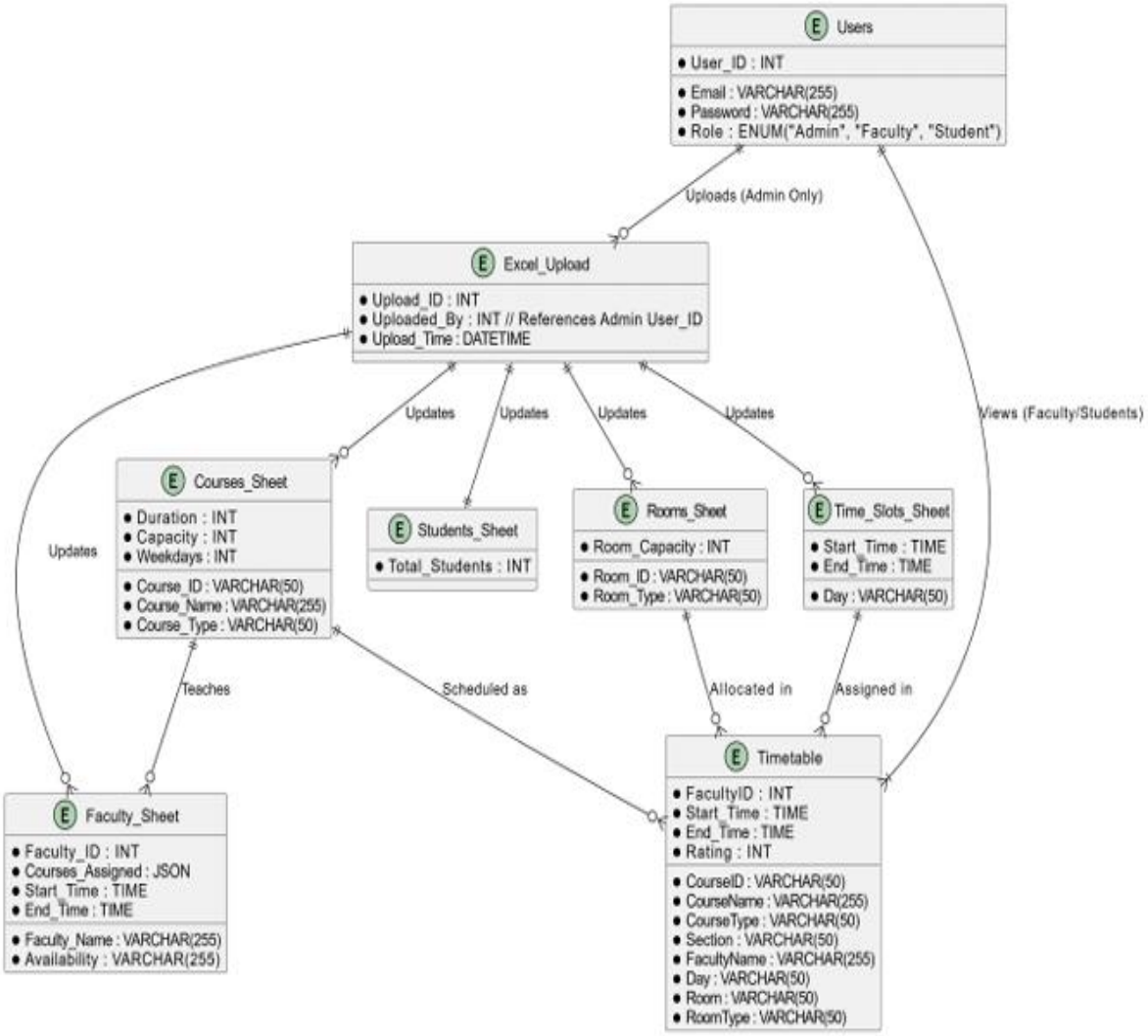

Figure 4-5: ERD

This ER diagram represents a **university timetable system** where:

- Users (Admin, Faculty, Students) authenticate using email and password.
- Admins can upload an Excel file containing course details, faculty availability, rooms, time slots, and student data.
- Timetable is generated based on the uploaded Excel file.
- Faculty and Students can only view the generated timetable.

5. METHODOLOGY

This chapter explains the process followed to develop the Smart Scheduler system. The system is designed to automate timetable creation using AI techniques, specifically Constraint Satisfaction Problems (CSP) and a Backtracking Search Algorithm. The methodology combines research-driven design with hands-on system development to address real-world academic scheduling issues. This approach allows the system to remain flexible, scalable, and capable of effectively managing resources within educational institutions.

5.1 Suggested Approach

The methodology behind Smart Scheduler is inspired by AI-based scheduling systems that address the complex requirements of academic timetabling. The project employs Constraint Satisfaction Problem (CSP) to model the scheduling problem, and utilizes a Backtracking algorithm to produce conflict-free and optimized academic timetables. The system manages both hard constraints (e.g. non-overlapping faculty or room assignments) and soft constraints (e.g. preferred time slots), producing high-quality and conflict-free schedules. The methodology is modular and flexible.

5.2 Design and Procedures

Adopts a constraint modeling and complete search approach using Backtracking to systematically find valid timetable solutions. The structured steps used in this project include:

- **Constraint Modeling**: Define and classify scheduling rules as hard or soft constraints.
- **Algorithm Selection**: Use a Backtracking algorithm to traverse the solution space, recursively assigning time slots while pruning invalid paths.
- **Testing Scenarios**: Use real and simulated scheduling data (courses, faculty, rooms) to validate timetable correctness and efficiency.
- **Performance Evaluation**: Measure response time, conflict resolution accuracy, and constraint satisfaction level.

5.3 Constraints Being Resolved

The CSP (Constraint Satisfaction Problem) solver handles the following constraints:

5.3.1 Hard Constraints

- **Room Type Matching**: Lab courses must be assigned to Lab rooms, Core courses to Lecture rooms.
- **Faculty Course Assignment**: Faculty can only teach courses they are assigned to.
- **Faculty Course Limit**: Each faculty member can teach maximum 3 courses.
- **Room Capacity**: Room capacity must accommodate course capacity.

- **No Double Booking**:
    - No room can be used by multiple courses at the same time.
    - No faculty can teach multiple courses at the same time.
    - No section can have multiple courses at the same time.
- **Course Duration**: Courses must fit within available time slots.
- **Course Frequency**: Courses must meet the required number of days per week.

5.3.2 Soft Constraints

- **Faculty Ratings**: Higher-rated faculty are preferred for courses.
- **Faculty Continuous Window**: Faculty's classes on a given day should be within a continuous time window.
- **Lecture Time Consistency**: All lectures for a section and faculty should be at the same start time each week.
- **Section Distribution**: Sections are automatically generated based on student capacity (V1, V2, etc. for small classes, C1, C2, etc. for larger classes)

5.4 Sources of Data and Sampling Procedures

The system uses realistic sample scheduling data from university to reflect common academic constraints and schedules. These include:

- Faculty availability and course load
- Classroom availability and capacities
- Institutional time slots and scheduling rules

5.4.1 Input Data Structure

The system expects an Excel file with 5 sheets containing the following data:

- **Courses Sheet**

  - **Course_ID**: Unique identifier (e.g., "CC141", "CC112L")
  - **Course_Name**: Course name (e.g., "Discrete Structures", "Object Oriented Programming Lab")
  - **Duration**: Time duration in human-readable format (e.g., "1 hour 15 minutes", "2 hours 30 minutes")
  - **Course_Type**: Either "Core" (lecture) or "Lab"
  - **Capacity**: Maximum students per section (e.g., 50)
  - **Weekdays**: Number of days per week the course meets (e.g., 2 for lectures, 1 for labs)

- **Faculty Sheet**

  - **Faculty_ID**: Unique identifier (integer)
  - **Faculty_Name**: Faculty member's name
  - **Courses_Assigned**: Dictionary of course assignments with ratings (e.g., {'CC112': 3, 'CC251': 5})
  - **Availability**: Days available (e.g., "Monday-Friday", "Tuesday-Thursday")
  - **Start_Time**: Earliest start time (integer, e.g., 8 for 8 AM)
  - **End_Time**: Latest end time (integer, e.g., 17 for 5 PM)

- **Rooms Sheet**

  - **Room_ID**: Unique room identifier (e.g., "Room1", "Room2")
  - **Room_Capacity**: Maximum capacity (e.g., 50)
  - **Room_Type**: Either "Lecture" or "Lab"

- **Time Slots Sheet**

  - **Day**: Day of the week (e.g., "Monday", "Tuesday")
  - **Start_Time**: Start time in 24-hour format (e.g., 8 for 8 AM)
  - **End_Time**: End time in 24-hour format (e.g., 18 for 6 PM)

- **Students Sheet**
  - **Total_Students**: Total number of students (e.g., 1300)

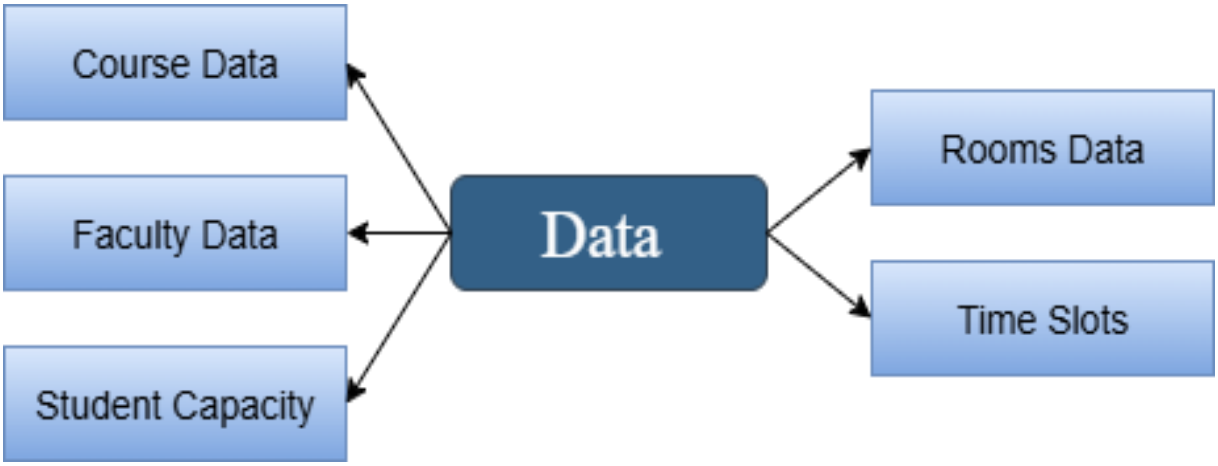

Figure 5-1: Data

5.5 Workflow of the System

The Smart Scheduler follows a systematic workflow that automates the scheduling process through the following steps:

- **Admin inputs data**: Faculty, departments, courses, and rooms are entered into the system.
- **Constraints are defined**: Scheduling rules are modeled as CSPs.
- **Backtracking algorithm runs**: The system recursively assigns time slots to classes while ensuring constraints are satisfied.
- **Conflict detection and resolution**: The system backtracks when conflicts arise and explores alternate paths.
- **Schedule generation**: A complete, valid timetable is generated upon successful assignment.
- **Users access output**: Faculty and students can view or download the finalized schedule via the UI.

5.6 Output Generated

The system produces an Excel file with the following structure:

5.6.1 Main Timetable Sheet

Organized by sections, containing:

- **CourseID**: Course identifier
- **CourseName**: Course name with section (e.g., "Discrete Structures-V1")
- **CourseType**: "Core" or "Lab"
- **Section**: Section identifier (e.g., "V1", "V2")
- **FacultyID**: Assigned faculty ID
- **FacultyName**: Assigned faculty name
- **Day**: Day of the week
- **Start Time**: Class start time (e.g., "8:00 AM")
- **End Time**: Class end time (e.g., "9:15 AM")
- **Room**: Assigned room
- **RoomType**: Room type
- **Rating**: Faculty rating for the course

5.6.2 Unassigned Courses Sheet(if any conflicts occur)

- **CourseName**: Course that couldn't be scheduled
- **Section**: Section identifier
- **Reason**: Detailed reason why the course couldn't be scheduled

5.7 Key Features

- **Automatic Section Generation**: Based on total students and course capacity
- **Faculty Rating System**: Prioritizes faculty based on their expertise ratings
- **Flexible Duration Handling**: Supports various time formats (1h, 1h15m, 1h30m, 2h30m)
- **Conflict Resolution**: Detailed reporting of scheduling conflicts
- **Timeout Protection**: Prevents infinite loops with 5-minute maximum runtime
- **Relaxed Mode**: Falls back to less strict constraints if initial scheduling fails

5.8 Limitations and Assumptions

5.8.1 Assumptions

- All scheduling data is accurate and provided in advance
- The academic calendar structure is fixed (e.g., number of days and time slots)

5.8.2 Limitations

- The Backtracking algorithm has exponential time complexity in the worst case, which may affect performance for extremely large institutions
- Soft constraints may not always be fully optimized if the system prioritizes hard constraints

6. IMPLEMENTATION DETAILS

6.1 Development Setup

The development of Smart Scheduler utilized a robust set of tools and technologies to ensure efficient, intelligent, and scalable timetable generation. The focus was on creating an automated scheduling system with strong AI capabilities and a responsive interface:

- **Programming Languages**:
  - Python was used for backend development and implementation of AI algorithms such as backtracking.
  - TypeScript was used for frontend development to ensure type safety and maintainability.
- **Frameworks**:
  - Next.js was adopted for building a fast, SEO-friendly, and interactive frontend.
  - FastAPI was used to develop high-performance APIs for backend communication.
- **Libraries**:
  - Pandas and NumPy were employed for data handling and manipulation.
  - PyMongo was used to interact seamlessly with the MongoDB database.
  - XlsxWriter to generate formatted Excel files for the final timetable.
- **Development Environment**:
  - Visual Studio Code (VS Code) was used as the primary IDE for coding, debugging, and project management.
- **Data Storage:**
  - CSV Files used for storing input data such as faculty preferences, course lists, and time slots.
- **Version Control**:
  - Git was utilized for version control and collaborative development, ensuring smooth team coordination.

6.2 Deployement Setup

Smart Scheduler was deployed using modern cloud-native tools to ensure smooth scalability, high performance, and continuous integration and delivery:

- **Containerization and Deployment**:
  - Docker was used to containerize the backend services for consistent and isolated deployment environments.
  - Railway served as the backend deployment platform, providing simple infrastructure management and CI/CD integration.
  - Vercel was used to deploy the frontend developed with Next.js, enabling fast, globally distributed access.
- **Database Hosting**:
  - The MongoDB database was hosted on a managed server to maintain data persistence and ensure secure access.

6.3 Algorithms

Smart Scheduler uses a mix of AI-driven and constraint satisfaction techniques to generate optimal class timetables. Below are the key algorithms:

6.3.1 Faculty Course Assignment Algorithm (Heuristic Rating-Based Selection)

- **Description:**

Selects the best 3 courses for each faculty member based on their rating preferences, ensuring fair and effective course load distribution.

- **Algorithm:**

```
Algorithm AssignTopThreeCourses(FacultyCourses, AllCourses):
   Input: FacultyCourses = {CourseID: Rating}, AllCourses = list of all CourseIDs
   Output: AssignedCourses = Top 3 Courses with Ratings

   1. Sort FacultyCourses by Rating in descending order
   2. If >= 3, return top 3
   3. Else, add sections of existing courses (if any)
   4. If still < 3, assign random courses with rating 1
   Return AssignedCourses
```

- **Time Complexity:**

O(n log n) for sorting, where *n* is the number of assigned courses per faculty.

6.3.2 Time Slot and Room Precomputation Algorithm

- **Description:**

Precomputes all valid time slot combinations and maps them to suitable room types to optimize room allocation during scheduling.

- **Algorithm:**

```
Algorithm PrecomputeTimeSlots(Schedule, Rooms, Courses):
   Input: Schedule days, Room list, Course list
   Output: ValidSlotMap

   For each day in Schedule:
      For each duration type (1h, 1h15m, 2h30m...):
         Generate all valid time blocks
         For each room:
            Map room to valid time blocks if capacity >= min(course.capacity)
   Return ValidSlotMap
```

- **Time Complexity:**

O(D × T × R) where D = number of days, T = time blocks, R = rooms

6.3.3 Constraint Bases Timetable Scheduling Algorithm(Bactracking + Heuristic)

- **Description:**

Uses backtracking to assign time slots, faculty, and rooms to courses, checking constraints like faculty load, room type, and conflict-free timing.

- **Algorithm:**

```
Algorithm ScheduleCourses(Courses):
   Input: List of Courses
   Output: Timetable

   If all courses are scheduled:
      Return Timetable
   For each course:
      For each faculty (sorted by rating):
         For each day:
            For each valid time slot:
               For each suitable room:
                  If no conflict (faculty, room, section, constraints):
                     Assign
                     Recurse to next course
                     If success, return
                     Else, backtrack
   Return Partial or Complete Timetable
```

- **Time Complexity:**

  Exponential in worst case

6.3.4 Conflict Detection Algorithm

- **Description:**

Ensures no overlapping assignments for faculty, rooms, or sections; checks constraints like continuous day windows and consistent lecture timing.

- **Algorithm:**

```
Algorithm IsConsistent(Assignment):
   Input: Proposed Time, Room, Faculty, Section
   Output: Boolean

   Check:
      - Room type matches course type
      - Faculty assigned < 3 courses
      - No overlapping time for room/faculty/section
      - Faculty schedule is within continuous window (optional)
      - Lectures of a course occur at same time weekly (optional)
   Return True if all constraints pass
```

- **Time Complexity:**

O(m) where *m* is the number of existing assignments to validate against.

6.3.5 Scheduling Timeout and Relaxed Retry Mechanism

- **Description:**

Prevents excessive computation on overly constrained courses by using timeouts and fallback retry with relaxed constraints.

- **Algorithm:**

```
Algorithm BacktrackWithTimeout(CourseIndex, TimeLimit):
   Start Timer
   While within TimeLimit:
      Try Assigning with Constraints
      If fail, retry with relaxed constraints
   If still fails:
      Skip and log conflict
```

- **Time Complexity:**

Controlled using per-course limit (default 5s) and overall scheduling limit (default 5 min).

6.4 Genetic Algorithm

The genetic algorithm is a metaheuristic optimization technique inspired by natural selection and genetics. For timetable generation, it treats each possible timetable as a "chromosome" and evolves better solutions over generations.

6.4.1 Code Structure and Flow

Below is the code structure and flow of the genetic algorithm.

6.4.1.1 Data Structures

```
@dataclass
class Gene:
    &quot;&quot;&quot;Represents a single session in the timetable&quot;&quot;&quot;
    CourseID: str
    Section: str
    FacultyID: int
    RoomID: str
    Day: str
    StartTime: float
    EndTime: float
@dataclass
class Chromosome:
    &quot;&quot;&quot;Represents a complete timetable solution&quot;&quot;&quot;
    genes: List[Gene]
    fitness: float = 0.0
```

- **Gene**: Represents one class session (e.g., CC141 Section A on Monday 8:00-9:30 in Room R1 with Faculty ID 44)
- **Chromosome**: Complete timetable containing all sessions (416 genes for your dataset)

6.4.1.2 Algorithm Flow

```
def run(self, output_file: str):
    &quot;&quot;&quot;Main execution flow&quot;&quot;&quot;
    self.step1_preprocess_inputs()      # Load and validate data
    self.step3_generate_initial_population()  # Create initial solutions
    self.step9_evolution_loop()         # Main optimization loop
```

self.step10_export_output(output_file) # Export results

- **Step 1: Data Preprocessing (step1_preprocess_inputs)**
  - **Loads CSV files**: Courses.csv, Faculty.csv, Rooms.csv, Time Slots.csv, Students.csv
  - **Validates data structure**: Ensures all required columns exist
  - **Processes faculty assignments**: Maps faculty to courses they can teach and Assigns ratings (1-5) based on expertise and ensures Faculty gets max 3 courses
  - **Generates time slots**: Creates 168 time slots (7 days × 24 hours)
  - **Validates constraints**: Checks room capacities, faculty availability
  - **Output:**
    - self.courses: List of course dictionaries with sections, durations, capacities
    - self.faculty: List of faculty with assigned courses and ratings
    - self.rooms: List of rooms with capacities and types (Lab/Lecture)
    - self.time_slots: List of available time slots
- **Step 2: Chromosome Structure Creation (step2_create_chromosome_structure)**
  - **Creates base genes**: For each course section, creates the required number of sessions
  - **Sets default values**: CourseID, Section, but leaves FacultyID, RoomID, Day, Time empty
  - **Ensures consistency**: All chromosomes will have the same structure (416 genes)
  - **Example:**

    Gene(CourseID="CC141", Section="A", FacultyID=0, RoomID="", Day="", StartTime=0.0, EndTime=0.0)
    Gene(CourseID="CC141", Section="A", FacultyID=0, RoomID="", Day="", StartTime=0.0, EndTime=0.0)

- **Fitness formula:**

  fitness = 1000 - (hard_penalties * 50) - (soft_penalties * 5)

- **Step 3: Initial Population Generation (step3_generate_initial_population)**
  - **For each base gene**:
    - **Faculty Assignment**: Selects available faculty for the course
    - **Room Assignment**: Finds suitable room (capacity, type)
    - **Time Assignment**: Picks available time slot
    - **Conflict Resolution**: Ensures no overlaps (room, faculty, section)
  - **Handles Core courses**: Ensures same faculty for both sessions
  - **Fallback mechanism**: Creates placeholder genes if constraints can't be satisfied
- **Step 4: Fitness Calculation (step4_calculate_fitness)**
  - **Hard Constraints (Heavy Penalties):**
    - **Faculty Assignment**: Faculty must be assigned to their courses
    - **Room Type**: Lab courses need lab rooms
    - **Room Capacity**: Room must fit course capacity
    - **No Conflicts**: No overlapping sessions (room, faculty, section)
    - **Faculty Load**: Faculty can't teach more than 3 courses
    - **Core Course Consistency**: Same faculty for both Core course sessions
  - **Soft Constraints (Light Penalties):**
    - **Faculty Preferences**: Preferred time slots
    - **Room Preferences**: Preferred room types
    - **Time Distribution**: Balanced schedule across days
    - **Gap Minimization**: Reduce gaps between classes
- **Step 5: Selection (step5_selection)**
  - **Tournament Selection Process:**
    - **Random Selection**: Pick tournament_size (3) random chromosomes
    - **Fitness Comparison**: Select the best one
    - **Repeat**: Until we have enough parents for crossover

- **Why Tournament Selection:**
  - Maintains diversity (not just top performers)
  - Allows some less fit chromosomes to reproduce
  - Prevents premature convergence

- **Step 6: Crossover (step6_crossover)**
  - **Crossover Process:**
    - **Alternating Selection**: Take genes alternately from parents

```
for i in range(min_length):
if i % 2 == 0:
child_genes.append(parent1.genes[i])  # Even indices from parent1
else:
child_genes.append(parent2.genes[i])  # Odd indices from parent2
```

    - **Length Handling**: If parents have different lengths, add remaining genes from longer parent

    - **Conflict Resolution**: Fix any conflicts in the child chromosome

  - **Example:**

```
Parent1:[Gene1,Gene2,Gene3,Gene4]
Parent2:[GeneA,GeneB,GeneC]
Child: [Gene1, GeneB, Gene3, Gene4] # Gene4 from parent1 (longer)
```

- **Step 7: Mutation (step7_mutation)**
  - **Mutation Types:**
    - **Time Mutation**: Change day/time slot
    - **Room Mutation**: Change room assignment
    - **Faculty Mutation**: Change faculty (except for Core courses)
  - **Mutation Rate**: 5% chance per gene (configurable)

- **Special Handling:**

  - **Core Courses**: Avoid faculty mutation to maintain same-faculty constraint
  - **Lab Courses**: Ensure lab rooms are maintained
  - **Capacity Constraints**: Ensure room capacity is sufficient

- **Step 8: Local Search (step8_local_search)**

  - **Local Search Process:**

    - **Gene Swapping**: Try swapping time slots between genes
    - **Fitness Check**: Calculate fitness before and after swap
    - **Acceptance**: Keep swap if fitness improves
    - **Iteration**: Repeat until no improvement or max iterations reached

- **Step 9: Evolution Loop (step9_evolution_loop)**

  - **Evolution Process:**

    - **Initial Fitness**: Calculate fitness for all chromosomes
    - **Generation Loop**: For each generation (150 max):

      i. **Selection**: Choose parents using tournament selection
      ii. **Crossover**: Create children by combining parents
      iii. **Mutation**: Randomly mutate some genes
      iv. **Fitness Calculation**: Evaluate new chromosomes
      v. **Elitism**: Keep best 10% of parents
      vi. **Population Update**: Replace population with children + elite

    - **Convergence Check**: Stop if no improvement for 20 generations
    - **Timeout Check**: Stop if time limit reached

- **Results:**

  Best Chromosome: Uses the chromosome with highest fitness

6.5 System Constraints

This section explains the system constraints in detail.

- **Faculty Course Limit Constraint**
  - Each faculty member can be assigned a maximum of 3 courses only.
  - This is enforced both in strict mode and during relaxed mode checks to prevent overloading instructors.
- **Room Type Constraint**
  - A Lab course must be assigned only to rooms with "Lab" type.
  - A Lecture course must be assigned only to rooms with "Lecture" type.
  - No cross-type assignment is allowed.
- **Room Capacity Constraint**
  - The selected room must have a capacity greater than or equal to the course's required capacity.
  - If no room meets the requirement, the course will be left unassigned.
- **No Double-Booking of Rooms**
  - A room cannot be assigned to two different courses during overlapping time slots.
- **No Time Overlap for Faculty**
  - A faculty member cannot be assigned to two classes that occur at the same time.
  - The algorithm checks all existing scheduled times for that faculty before assigning a new course.
- **No Time Overlap for Sections**
  - A course section cannot be assigned two classes at the same time.
- **Faculty Continuous Time Window Constraint (Per Day)**
  - All classes assigned to a faculty member on a given day must fall within one continuous window (no long gaps between lectures).
  - For example, if a faculty teaches from 9:00–10:00 and then again at 3:00–4:00, it violates this constraint.

- **Lecture Fixed Timing Constraint (Weekly Consistency)**
  - Lectures for the same course-section-faculty must start at the same time each day across the week.
  - If the lecture starts at 9:30 AM on Monday, it must also start at 9:30 AM on Wednesday or Friday.
- **Faculty-Course Match Constraint**
  - Only those courses included in a faculty member's Processed_Courses list (rated and filtered to 3) are eligible for assignment.
  - Even in relaxed mode, courses without a reasonable match may be skipped if all constraints fail.
- **Time Slot Fit Constraint**

  The algorithm selects a time slot that matches the course's duration:
  - Courses are grouped by duration like 1h, 1h15m, 1h30m, or 2h30m.
  - It ensures that time blocks assigned match this duration type.
- **Course Weekday Constraint**
  - Each course must be assigned to run on a specific number of weekdays (e.g., 2 or 3), based on the Weekdays field in the course data.
  - These days must be distributed to avoid too-close back-to-back scheduling (e.g., must have 2-day gaps).
- **Scheduling Timeout Constraint**
  - Each course is allowed a maximum of 5 seconds (MAX_TIME_PER_COURSE) for scheduling.
  - The total scheduling process must complete within 5 minutes (MAX_SCHEDULING_TIME).
  - If these limits are exceeded, the course may be skipped and logged as a conflict.

- **Relaxed Mode Constraint Handling**

  If the normal constraint-based scheduling fails, the system enters a "relaxed mode" where it:

  - Allows course assignment outside the faculty's top-rated 3
  - Relaxes some timing consistency

  Even in relaxed mode, major constraints like room type and no-overlap are still enforced.

6.6 Assumptions

- Faculty members are available for the number of days and time slots required by their assigned courses.
- Room capacities are assumed accurate and suitable for assigned courses.
- Each course has complete and correct metadata (duration, weekdays, type).
- Faculty will teach only 3 courses maximum.
- All input files (Courses, Faculty, Rooms, Time Slots) are correctly formatted and validated.

6.7 Restrictions

- Currently limited to working with CSV-based inputs only, it assumes files like Faculty.csv, Courses.csv, etc.
- Faculty-course relationships are rating-based, so incorrect or missing ratings may affect scheduling.
- Room assignment only considers room type and capacity, not features like equipment or location.

6.8 Limitations

- Does not support dynamic rescheduling or manual override after initial timetable generation.
- Does not incorporate faculty leave, holidays, or external unavailability.
- No machine learning used — it's rule-based and heuristic AI (may limit adaptability over time).

7. TESTING

7.1 Extended Test Cases

The following tables outlines detailed test cases designed to cover extended scenarios beyond standard functionality.

Test Case ID: TC-SS-001						
Test Module Name: User Registration						
Test Priority: High						
Test Title/Name: Register with Email Verification						
Description: Verify that a user cannot register without verifying email using a 4-digit code.						
Pre-Condition: User must have a valid email address.						
Dependencies: Email service, API for registration and code verification.						
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status (Pass/Fail)**	**Notes**
1	Navigate to Signup Page	-	Signup page is displayed	-	Pass	-
2	Enter valid details	Name, Email, Password, Role	Verification code is sent to email	-	Pass	-
3	Enter incorrect or no code	-	Error: Enter valid code	-	Pass	Cannot Proceed
4	Enter correct code and click Verify	4-digit code	Email verified, message shown	-	Pass	-
5	Click Resend Code	-	New code sent via email	-	Pass	-
Post Condition: User is registered only after successful email verification.						

Test Case ID: TC-SS-002						
Test Module Name: User Login						
Test Priority: High						
Test Title/Name: Role-Based Login Handling						
Description: Ensure user is redirected based on role or shown rejection/pending messages.						
Pre-Condition: User must be registered.						
Dependencies: Login API, user status logic.						
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status (Pass/Fail)**	**Notes**
1	Open login page	-	Login form visible	-	Pass	-
2	Open login page	Email, Password	Redirected to dashboard if approved	-	Pass	-
3	Enter credentials	Pending user	Message: Your request is pending	-	Pass	-
4	Enter credentials	Rejected user	Message: Not eligible	-	Pass	-
Post Condition: User is logged in or shown correct status message.						

Test Case ID: TC-SS-003						
Test Module Name: Admin Login						
Test Priority: High						
Test Title/Name: Admin Credential Validation						
Description: Verify that admin only logs in with valid credentials.						
Pre-Condition: Admin credentials must be created.						
Dependencies: Admin login API.						
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status (Pass/Fail)**	**Notes**
1	Enter invalid credentials	Wrong Email/Password	Message: Invalid credentials	-	Pass	-
2	Enter valid credentials	Admin Email, Password	Redirect to admin dashboard	-	Pass	-
Post Condition: Admin reaches dashboard only with correct login.						

Test Case ID: TC-SS-004						
Test Module Name: Admin Dashboard Cards						
Test Priority: Medium						
Test Title/Name: Dashboard User Stats Display						
Description: Validate correct display of total users, accepted, rejected, and pending counts.						
Pre-Condition: Users must exist in various statuses.						
Dependencies: Dashboard API						
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status (Pass/Fail)**	**Notes**
1	Load dashboard	-	Cards with correct counts shown	-	Pass	-
2	Verify table	-	Pending user table is loaded properly	-	Pass	-
Post Condition: Admin sees correct statistics and pending user list.						

Test Case ID: TC-SS-005						
Test Module Name: Admin Approval						
Test Priority: High						
Test Title/Name: Approve/Reject Users						
Description: Admin can approve/reject and user gets notified.						
Pre-Condition: At least one pending user request.						
Dependencies: Approval API, Email service						
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status (Pass/Fail)**	**Notes**
1	Click Approve	Pending user	User marked as approved + email sent	-	Pass	-
2	Click Reject	Pending user	User marked as rejected + email sent	-	Pass	-
Post Condition: User receives approval/rejection notification.						

Test Case ID: TC-SS-006						
Test Module Name: User History Edit						
Test Priority: Medium						
Test Title/Name: Edit User Role or Status						
Description: Admin can modify user role or status from history tab.						
Pre-Condition: User list must be populated.						
Dependencies: Role update API						
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status (Pass/Fail)**	**Notes**
1	Click Edit	User Row	Role/Status becomes editable	-	Pass	-
2	Change Values	Role, Status	Dropdowns shown, values changed	-	Pass	-

Post Condition: User details are successfully updated.						
Test Case ID: TC-SS-007						
Test Module Name: Upload Excel Timetable						
Test Priority: High						
Test Title/Name: Upload and Validate File						
Description: Admin uploads and validates Excel file for scheduling.						
Pre-Condition: Valid Excel file exists.						
Dependencies: Upload and validate APIs						
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status (Pass/Fail)**	**Notes**
1	Upload file	Faculty.xlsx	File uploaded successfully	-	Pass	-
2	Click Validate	-	File uploaded successfully	-	Pass	-
Post Condition: Files are uploaded and validated.						

Test Case ID: TC-SS-008						
Test Module Name: Timetable Generation						
Test Priority: High						
Test Title/Name: Generate Timetable Using CSP						
Description: Validate backend generates timetable using CSP successfully.						
Pre-Condition: Valid files uploaded.						
Dependencies: Backend CSP logic (e.g., csp3.py)						
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status (Pass/Fail)**	**Notes**
1	Click Generate	-	Timetable is generated	-	Pass	-
2	Download file	-	File downloaded	-	Pass	-
Post Condition: Timetable ready to use/download.						

Test Case ID: TC-SS-009						
Test Module Name: Generated Timetable Tab						
Test Priority: Medium						
Test Title/Name: Download or Delete Timetable						
Description: Admin can download or delete generated timetable.						
Pre-Condition: Timetable exists in database.						
Dependencies: Download/Delete API						
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status (Pass/Fail)**	**Notes**
1	Click Download	File name	File downloaded	-	Pass	-
2	Click Delete	File name	File removed from list	-	Pass	-
Post Condition: Admin manages timetable files easily.						

Test Case ID: TC-SS-010						
Test Module Name: Email Code Resend						
Test Priority: Medium						
Test Title/Name: Resend 4-digit Verification Code						
Description: Verify that the user can request a new code.						
Pre-Condition: Registration form filled but code expired.						
Dependencies: Email resend API						
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status (Pass/Fail)**	**Notes**
1	Click Resend Code	-	New code sent to email	-	Pass	-
2	Enter new code	4-digit code	Email verification succeeds	-	Pass	-
Post Condition: User receives new code and verifies email.						

7.2 Decision Table

The following sections provide a detailed breakdown of logic, requirements, use cases, and test mappings to validate system functionality through decision-based testing.

7.2.1 Code Snippet

```
import logging
from datetime import datetime

logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(levelname)s - %(message)s")

def register_user(is_email_entered, is_password_entered, is_code_verified):
    if not is_email_entered or not is_password_entered:
        return "Error: Required Fields Missing"
    if not is_code_verified:
        return "Error: Email Not Verified"
    return "Registration Successful"

def login_user(is_registered, is_approved, correct_password, is_verified):
    if not is_registered:
        return "Not Registered"
    if not is_approved:
        return "Request Pending or Rejected"
    if not correct_password:
        return "Incorrect Password"
    if not is_verified:
        return "Verification Required"
    return "Login Successful"

def validate_excel(file_uploaded, file_valid_format):
    if not file_uploaded:
        return "No File Uploaded"
    if not file_valid_format:
        return "Invalid File Format"
    return "File Validated"

def generate_timetable(file_validated, schedule_possible):
    if not file_validated:
        return "File Validation Required"
    if not schedule_possible:
        return "Generation Failed"
    return "Timetable Generated"
```

7.2.2 Decision Coverage Table

The table below outlines condition-based decisions and their expected outcomes for key system functions.

Function	Condition 1	Condition 2	Condition 3	Condition 4	Decision	Outcome
Register User	Email Entered	Password Entered	Code Verified	-	All Valid	Registration Successful
Register User	Not Entered	Any	Any	-	Missing Email	Error
Register User	Entered	Not Entered	Any	-	Missing Password	Error
Register User	Entered	Entered	Not Verified	-	Code Missing	Error
Login User	Registered	Approved	Correct Password	Verified	All Valid	Login Successful
Login User	Not Registered	Any	Any	Any	Not Registered	Login Failed
Login User	Registered	Not Approved	Any	Any	Approval Missing	Login Failed
Login User	Registered	Approved	Wrong Password	Any	Incorrect Password	Login Failed
Login User	Registered	Approved	Correct Password	Not Verified	Verification Needed	Login Failed
Validate Excel	Uploaded	Valid Format	-	-	All Valid	File Validated
Validate Excel	Not Uploaded	Any	-	-	File Missing	Error
Validate Excel	Uploaded	Invalid Format	-	-	Format Issue	Error
Generate Timetable	File Validated	Scheduling Possible	-	-	All Valid	Timetable Generated
Generate Timetable	Not Validated	Any	-	-	File Invalid	Error
Generate Timetable	Validated	Scheduling Not Possible	-	-	Conflict	Error

Table 7-1: Decision Coverage Table

7.2.3 RID Definitions

The table lists system requirements with unique identifiers and their corresponding descriptions.

RID	Requirement Description
RID-001	User Registration with verification
RID-002	User Login with role validation and approval
RID-003	Upload Excel functionality
RID-004	Excel format and data validation
RID-005	Timetable generation based on valid input
RID-006	Timetable download functionality
RID-007	Admin approval and rejection for requests
RID-008	Admin request listing and history management
RID-009	Email resend and verification handling
RID-010	Delete generated timetable and edit history

Table 7-2: RID Definitions

7.2.4 UCID Definitions

The table below defines use cases that describe key user interactions with the system.

UCID	Use Case Description
UC-1	Register new user (email, username, password, role, and verify code)
UC-2	Login user with credentials and role approval
UC-3	Upload Excel sheet with scheduling data
UC-4	Validate uploaded Excel format and contents
UC-5	Generate timetable based on validated data
UC-6	Download timetable as output
UC-7	Admin approves or rejects registration requests
UC-8	Admin views request history and logs
UC-9	Resend verification code or verify code
UC-10	Admin deletes or edits timetable and activity logs

Table 7-3: UCID Definitions

7.2.5 TID Definitions

The table outlines test cases identified by unique IDs, each designed to verify specific system behaviors.

TID	Test Case Description
TC-001	Verify registration with valid email, username, password, role, and code
TC-002	Verify error message on missing fields during registration
TC-003	Verify login with valid credentials and approval
TC-004	Upload Excel file with valid format
TC-005	Upload Excel file with invalid format
TC-006	Validate Excel structure and required columns
TC-007	Reject validation on empty or malformed data
TC-008	Generate timetable after successful validation
TC-009	Fail timetable generation when conflicts exist
TC-010	Download generated timetable
TC-011	Admin approves pending request
TC-012	Admin rejects request
TC-013	Admin views full request history
TC-014	Admin clears or edits history logs
TC-015	Resend verification code to email
TC-016	Verify email with 4-digit code
TC-017	Admin deletes specific timetable
TC-018	Admin edits action logs

Table 7-4: TID Definitions

7.2.6 RID vs UCID (Requirements vs Use Cases)

The matrix below maps each system requirement (RID) to its corresponding use cases to ensure full functional coverage.

RID (Requirement ID)	UC-1	UC-2	UC-3	UC-4	UC-5	UC-6	UC-7	UC-8	UC-9	UC-10
RID-001	✓									
RID-002		✓								
RID-003			✓							
RID-004			✓	✓						
RID-005				✓	✓					
RID-006					✓	✓				
RID-007							✓			
RID-008							✓	✓		
RID-009									✓	
RID-010										✓

Table 7-5: RID vs UCID matrix

7.2.7 RID vs TID (Requirements vs Test Case ID)

The following table links each requirement to its related test cases to validate implementation and functionality.

RID (Requirement ID)	TID (Test Case ID)
RID-001	TC-001, TC-002
RID-002	TC-003
RID-003	TC-004, TC-005
RID-004	TC-006, TC-007
RID-005	TC-008, TC-009
RID-006	TC-010
RID-007	TC-011, TC-012
RID-008	TC-013, TC-014
RID-009	TC-015, TC-016
RID-010	TC-017, TC-018

Table 7-6: RID vs TID

7.2.8 UCID vs TID (Use Cases vs Test Case ID)

The table below maps each use case to its corresponding test cases to ensure thorough testing of user interactions.

UCID (Use Case ID)	TID (Test Case ID)
UC-1	TC-001, TC-002
UC-2	TC-003
UC-3	TC-004, TC-005
UC-4	TC-006, TC-007
UC-5	TC-008, TC-009
UC-6	TC-010
UC-7	TC-011, TC-012
UC-8	TC-013, TC-014
UC-9	TC-015, TC-016
UC-10	TC-017, TC-018

Table 7-7: UCID vs TID

8. RESULTS

8.1 Completion

From the RID vs UCID matrix:

- Total Requirements (RID): 10
- Mapped Requirements: 10
- % Completion = 100%

8.2 Accuracy

From the RID vs TID matrix:

- Total Requirements (RID): 10
- Requirements Verified with Test Cases: 10
- % Accuracy = 100%

8.3 Correctness

From the UCID vs TID matrix:

- Total Use Cases (UCID): 10
- Use Cases Mapped to Test Cases: 10
- % Correctness = 100%

9. CONCLUSION

The Smart Scheduler project implements a powerful automated academic timetable scheduling process through a combination of AI and constraint satisfaction methods. The system provides educational institutions with a flexible solution through its ability to handle resource distribution requirements together with teacher availability and class scheduling problems.

The system uses enhanced optimization algorithms to optimize timetabling processes and lower administrative burden and create fair highly efficient schedules. Throughout their implementation AI-based methods boost organizational efficiency while helping schools efficiently deal with changing circumstances and staying within their operational boundaries.

The academic scheduling evolution reached its key milestone with the implementation of the Smart Scheduler. The project helps achieve educational administrative goals through technological improvements that make timetables more efficient and reliable and accessible for all administrators. The scheduling performance could be improved through enhancements that would include implementing real-time adjustments together with cloud-based accessibility and additional AI-driven optimizations.

10. FUTURE WORK

The Smart Scheduler has successfully demonstrated the capability to generate conflict-free, resource-optimized timetables, several areas remain open for further enhancement.

A key direction is the development of a standalone desktop application to increase accessibility and deployment flexibility across institutional environments.

Additionally, integrating a dynamic soft constraint input mechanism would allow faculty members to specify their scheduling preferences improving user satisfaction and personalization. The current use of a Genetic Algorithm is effective in handling complex scheduling constraints but exhibits non-deterministic behavior.

We will focus on refining this approach to improve accuracy and consistency in the generated timetables. Moreover, while the backtracking algorithm ensures optimal results, its time complexity causes scalability challenges. Optimization techniques can be explored to reduce computational overhead and enhance performance for large-scale scheduling scenarios.

11. BIBLIOGRAPHY

11.1 Books

1. P. Brucker, *Scheduling Algorithms*, 5th ed. Berlin, Germany: Springer, 2007

2. F.Werner, L. Burtseva, and Y. Sotskov, *Algorithms for Scheduling Problems*. MDPI, 2018.

11.2 Journals

1. J. Y.-T. Leung, Ed., *Handbook of Scheduling: Algorithms, Models, and Performance Analysis*, 1st ed. New York, NY, USA: Chapman and Hall/CRC, 2004. doi: 10.1201/9780203489802.

11.3 Articles

1. B. Sigl, M. Golub, and V. Mornar, "Solving timetable scheduling problem by using genetic algorithms

11.4 Research papers

1. Ansari, A. and Bojewar, S. 2014. Genetic algorithm to generate the automatic timetable – An overview. *Int. J. Recent Innov. Trends Comput. Commun.*

2. El-Sakka, T. 2015. University course timetable using constraint satisfaction and optimization. *Int. J. Comput. Acad. Res.*

3. Sayed, S. et al. 2015. Automated timetable generator. *Int. J. Innov. Res. Sci. Technol.*

4. Mittal, D., Doshi, H., Sunasra, M., and Nagpure, R. 2015. Automatic timetable generation using genetic algorithms. *Int. J. Adv. Res. Comput. Commun. Eng.*

5. Ambhore, S. et al. 2020. Automated timetable generation using machine learning algorithms. *Int. J. Res. Eng. Sci. Manag.*

6. Markal, S., Ghorpade, S., and Chalke, D. 2020. Timetable generator. *IOSR J. Comput. Eng.*

7. Müller, T. 2024. Student scheduling at Purdue University. In *Proceedings of the PATAT 2024 Conference*.

12. APPENDIX

12.1 Glossary of Terms

The glossary below defines key terms and abbreviations used throughout the tables for clarity and reference.

Term	Explanation
NP-Hard	A classification of problems that are highly complex and computationally intensive.
Docker	A platform for containerizing applications for easier deployment.
Azure	Microsoft's cloud computing service for hosting and AI development.
Git	A version control system used for tracking changes in source code.
tqdm	A Python tool for displaying progress bars during code execution.
Heuristic Search	A problem-solving technique that employs practical methods to find satisfactory solutions more quickly.
Encryption	A method of securing data by converting it into an unreadable format that can only be deciphered with a key.
Containerization	A lightweight method of virtualizing applications, allowing them to run in isolated environments (e.g., using Docker).

Table 12-1: Glossary